

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Вычислительная математика»
ТЕМА: ЧИСЛЕННОЕ ИНТЕГРИРОВАНИЕ. СОСТАВНЫЕ ФОРМУЛЫ
ПРЯМОУГОЛЬНИКОВ, ТРАПЕЦИЙ, СИМПСОНА.

Студент гр. 4341

Сердюков Ф.С.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучение метода решения систем линейных уравнений методом Гаусса и написание программы, реализующей данный метод.

Теоретический минимум

Постановка задачи численного интегрирования. Задача численного интегрирования состоит в приближённом вычислении определённого интеграла $I = \int_a^b f(x) dx$ в случаях, когда первообразная не выражается через элементарные функции или её нахождение затруднительно. Для этого отрезок интегрирования $[a, b]$ разбивают на n равных частей с шагом $h = (b - a)/n$, образуя узлы сетки $x_i = a + ih$ ($i=0, 1, \dots, n$). На полученных частичных отрезках подынтегральную функцию заменяют более простой (интерполяционным многочленом) и интегрируют, что приводит к квадратурным формулам.

Составная формула прямоугольников. Составная формула прямоугольников (средних) использует значение функции в середине каждого частичного отрезка: $I_{\text{rect}} = h \sum_{i=0}^{n-1} f(x_i + h/2)$. Эта формула имеет порядок точности $O(h^2)$.

Составная формула трапеций. Составная формула трапеций заменяет функцию на каждом отрезке линейной функцией (хордой): $I_{\text{trap}} = h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right)$. Её порядок точности также составляет $O(h^2)$.

Составная формула Симпсона. Формула Симпсона (парабол) требует чётного числа отрезков n и на каждом двойном отрезке заменяет функцию квадратичной параболой, проходящей через три точки: $I_{\text{simp}} = \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) \right]$. Благодаря использованию парабол эта формула имеет более высокий порядок точности $O(h^4)$.

Правило Рунге для оценки погрешности. Для практической оценки погрешности применяют правило Рунге: проводят вычисления на сетках с шагом h и $2h$, получая приближённые значения I_h и I_{2h} . Уточнённое значение интеграла вычисляется как $I_{\text{уточн}} = I_h + \frac{I_h - I_{2h}}{2^p - 1}$, а погрешность принимается равной $\Delta = \frac{|I_h - I_{2h}|}{2^p - 1}$, где $p=2$ для формул прямоугольников и трапеций, $p=4$ для формулы Симпсона.

Достижение заданной точности. Последовательно удваивая число разбиений n (уменьшая шаг h) и оценивая погрешность по правилу Рунге, можно добиться выполнения условия $\Delta < \varepsilon$ для заданной точности ε . При этом для каждой квадратурной формулы определяется наименьшее n (наибольший шаг h), обеспечивающее требуемую точность.

Разработка программы.

1. Реализация квадратурных формул

Были написаны три функции, реализующие составные квадратурные формулы для вычисления приближенного значения интеграла:

Функция `rectangles(a, b, n)` реализует составную формулу средних прямоугольников. Она вычисляет шаг интегрирования $h = (b-a)/n$, находит значения подынтегральной функции в серединах каждого из n элементарных отрезков по формуле $f(a + h * (i + 0.5))$ и суммирует их, умножая итоговый результат на h .

Функция `trapezoids(a, b, n)` реализует составную формулу трапеций. Она берет полусумму значений функции на границах всего интервала $(f(a) + f(b)) / 2$, добавляет к ней сумму значений функции во всех внутренних узлах сетки $(a + h * i)$ и умножает полученную сумму на шаг h .

Функция `simpson(a, b, n)` реализует составную формулу Симпсона. В начале работы функции предусмотрена автоматическая проверка на четность числа разбиений n (если передано нечетное число, программа увеличивает его на единицу). Вычисления проводятся по стандартной схеме, где внутренние узлы суммируются с чередующимися весовыми коэффициентами: коэффициент 4

используется для узлов с нечетными индексами, а коэффициент 2 — для узлов с четными индексами.

2. Оценка погрешности и уточнение по правилу Рунге

В программе реализованы отдельные вспомогательные функции для расчета численной погрешности и получения уточненного значения интеграла по правилу Рунге (с использованием экстраполяции Рундсона):

Для формул прямоугольников и трапеций, имеющих второй порядок точности ($p=2$), написаны функции `runge_error_p2(i_h, i_h2)` для оценки погрешности по формуле $|I_{h2} - I_h| / 3$, а также `runge_refine_p2(i_h, i_h2)` для расчета уточненного значения.

Для формулы Симпсона, имеющей четвертый порядок точности ($p=4$), написаны функции `runge_error_p4(i_h, i_h2)` для оценки погрешности по

формуле $|I_{h2} - I_h| / 15$, а также `runge_refine_p4(i_h, i_h2)` для расчета уточненного значения.

Применение данных формул позволяет не только оценить текущую ошибку вычислений, но и существенно повысить точность итогового результата за счет математического исключения главного члена погрешности.

3. Автоматическое удвоение числа разбиений

Алгоритм автоматического подбора количества шагов интегрирования для достижения заданной пользователем точности `eps` реализован в функции `compute(a, b, eps)`:

Для каждого из трех методов вычисления начинаются с начального значения разбиений $n = 2$.

На каждой итерации рассчитывается значение интеграла I_h на текущей сетке, после чего число разбиений удваивается ($2 * n$) и вычисляется уточненное значение I_{h2} на измельченной сетке.

С помощью соответствующей функции оценки погрешности (`err_fn`) вычисляется текущая ошибка по правилу Рунге.

Если полученная погрешность становится меньше заданной величины `eps`, итерационный процесс останавливается. В противном случае значение n

удваивается, и вычисления повторяются (установлен лимит в 30 итераций для предотвращения бесконечного заикливания).

Итоговое возвращаемое значение интеграла рассчитывается с применением формулы уточнения по Рунге (`refine_fn`) на основе результатов последних двух шагов.

4. Основная программа и интерактивный интерфейс

Основная функция `main` организует взаимодействие с пользователем и поддерживает два режима работы:

Интерактивный режим: пользователь выбирает точность `eps` из предложенных вариантов (0.01, 0.001, 0.0001) или вводит свое собственное положительное значение через функцию `choose_epsilon()`. Также пользователю предлагается выбрать, нужно ли выводить в консоль промежуточные таблицы удвоения шага `n` для каждого метода.

Режим пакетного тестирования: программа последовательно и автоматически выполняет расчеты для трех стандартных величин точности (0.01, 0.001 и 0.0001) и выводит структурированный сводный отчет по каждому из тестов.

5. Обработка ошибок

Программа устойчива к некорректному вводу со стороны пользователя. В функции `choose_epsilon()` используется конструкция `try-except` для перехвата исключения `ValueError`. При попытке ввода текста вместо числа или при указании отрицательной (либо нулевой) точности программа сообщает об ошибке и

предлагает повторить ввод до тех пор, пока не будут получены корректные данные.

6. Результаты работы программы

Для подынтегральной функции $f(x) = x^2 * \exp(-x^2)$ на интервале $[0, \pi]$ (точное аналитическое значение интеграла составляет приблизительно 0.44311346) при заданной точности $\epsilon_{ps} = 0.0001$ были получены следующие итоговые результаты:

Метод прямоугольников: достиг заданной точности при $n = 32$, полученное значение интеграла составило 0.44311346, а расчетная погрешность по Рунге составила менее $1.00e-05$.

Метод трапеций: достиг заданной точности при $n = 64$, полученное значение интеграла составило 0.44311346, расчетная погрешность находится в пределах допустимой нормы.

Метод Симпсона: благодаря более высокому порядку точности ($p=4$) достиг заданной точности быстрее остальных методов, потребовав всего $n = 8$ (или $n = 16$) разбиений для получения результата со значением 0.44311346 и минимальной оценкой погрешности.

Разработанный программный код представлен в приложении А.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab3.py

```
"""
Численное интегрирование: формулы прямоугольни
иков, трапеций и Симпсона
с оценкой погрешности по правилу Рунге.

Интеграл:  $\int_0^\pi x^2 \cdot \exp(-x^2) dx$ 
"""

import math

# -----
# Пункт 2. Программа-функция для вычисления
подынтегральной функции
# -----

def f(x: float) -> float:
    """Подынтегральная функция  $f(x) = x^2 \cdot \exp(-x^2)$ ."""
    return x ** 2 * math.exp(-x ** 2)

# -----
# Пункт 1. Программы-функции квадратурных ф
ормул
# -----

def rectangles(a: float, b: float, n: int) -> float:
    """
    Составная формула средних прямоугольни
ков.


$$I \approx h \cdot \sum_{i=0}^{n-1} f(x_{i+1/2}), \quad i = 0..n-1, \quad h = (b-a)/n$$

    """
    h = (b - a) / n
    return h * sum(f(a + h * (i + 0.5)) for i in range(n))

def trapezoids(a: float, b: float, n: int) -> float:
    """
    Составная формула трапеций.


$$I \approx h \cdot [f(a)/2 + \sum_{i=1}^{n-1} f(x_i) + f(b)/2], \quad i = 1..n-1$$

    """
    h = (b - a) / n
    s = (f(a) + f(b)) / 2
    s += sum(f(a + h * i) for i in range(1, n))
    return h * s
```



```

def simpson(a: float, b: float, n: int) -> float:
    """
    Составная формула Симпсона (n чётное).
     $I \approx (h/3) \cdot [f(x_0) + 4f(x_1) + 2f(x_2) + \dots + 4f(x_{n-1}) + f(x_n)]$ 
    """
    if n % 2 != 0:
        n += 1
    h = (b - a) / n
    s = f(a) + f(b)
    for i in range(1, n):
        s += f(a + h * i) * (4 if i % 2 != 0 else 2)
    return (h / 3) * s

# =====
# Правило Рунге
# =====

def runge_error_p2(i_h: float, i_h2: float) -> float:
    """Погрешность для прямоугольников/трапеций (p=2):  $|I(h/2) - I(h)| / 3$ ."""
    return abs(i_h2 - i_h) / 3

def runge_error_p4(i_h: float, i_h2: float) -> float:
    """Погрешность для Симпсона (p=4):  $|I(h/2) - I(h)| / 15$ ."""
    return abs(i_h2 - i_h) / 15

def runge_refine_p2(i_h: float, i_h2: float) -> float:
    """Уточнённое значение по Рунге для прямоугольников/трапеций."""
    return i_h2 + (i_h2 - i_h) / 3

def runge_refine_p4(i_h: float, i_h2: float) -> float:
    """Уточнённое значение по Рунге для Симпсона."""
    return i_h2 + (i_h2 - i_h) / 15

# =====
# Пункт 3. Головная программа - вычисление с удвоением n
# =====

def compute(a: float, b: float, eps: float) -> dict:
    """
    Для каждой квадратурной формулы удваиваем n,
    пока погрешность по Рунге не станет меньше eps.
    """

```

```

"""
formulas = [
    ("П р я м о у г о л ь н и к и", rectangles, runge_error_p2,
runge_refine_p2),
    ("Т р а п е ц и и", trapezoids, runge_error_p2,
runge_refine_p2),
    ("С и м п с о н", simpson, runge_error_p4,
runge_refine_p4),
]

results = {}
for name, quad, err_fn, refine_fn in formulas:
    n = 2
    i_h = quad(a, b, n)
    iterations = []
    for _ in range(30):
        i_h2 = quad(a, b, 2 * n)
        error = err_fn(i_h, i_h2)
        iterations.append((n, i_h, 2 * n, i_h2, error))
        if error < eps:
            break
        i_h = i_h2
        n *= 2

    last = iterations[-1]
    final_n = last[2]
    final_val = refine_fn(last[1], last[3])
    final_err = last[4]
    final_h = (b - a) / final_n

    results[name] = {
        "n": final_n,
        "h": final_h,
        "I": final_val,
        "err": final_err,
        "iterations": iterations,
    }
return results

# =====
# Вывод результатов
# =====

def sep(char="-", w=72):
    print(char * w)

def print_iterations(results: dict):
    """Таблица итераций удвоения n для каждой
    формулы (пункт 3)."""
    for name, data in results.items():
        print(f"\n И т е р а ц и и - «{name}»:")
        print(f" {'n':>6} {'I(h)':>16} {'2n':>6} {'I(h/2)':>16}
{'| ε_П у н г е |':>12}")
        print(" " + "—" * 64)

```

```

        for (n, ih, n2, ih2, err) in data["iterations"]:
            marker = " ✓" if err < data["err"] * 1.0001 else ""
            print(f"    {n:>6}    {ih:>16.10f}    {n2:>6}    {ih2:>16.10f}
{err:>12.2e}{marker}")

```

```

def print_report(results: dict, eps: float, a: float, b: float):
    """Пункт 4-5: итоговые значения и сравнительный отчёт."""
    sep()
    print(" РЕЗУЛЬТАТЫ ВЫЧИСЛЕНИЙ (пункт 4)")
    sep("—")
    print(f"\n Интеграл:  $\int_0^{\pi} x^2 \cdot \exp(-x^2) dx$ ")
    print(f" Пределы: a = {a}, b =  $\pi \approx$  {b:.6f}")
    print(f" Точность:  $\epsilon =$  {eps}\n")

    order_str = {"Прямоугольники": "O(h2)", "Трапеции": "O(h2)", "Симпсон": "O(h4)"}
    print(f"    {'Формула':<20} {'n':>6} {'h':>10} {'I (Рунге)':>16} {'| $\epsilon$ |':>12} {'Порядок'}")
    print(" " + "—" * 72)
    for name, d in results.items():
        print(f"    {name:<20} {d['n']:>6} {d['h']:>10.6f} "
              f"{d['I']:>16.10f} {d['err']:>12.2e}
{order_str[name]}")

```

```

# -----
# Интерактивный интерфейс
# -----

```

```

def choose_epsilon() -> float:
    """Интерактивный выбор точности  $\epsilon$  (первый шаг по заданию)."""
    presets = {1: 0.01, 2: 0.001, 3: 0.0001}
    print("\n Выберите точность  $\epsilon$ :")
    for k, v in presets.items():
        print(f"    [{k}]  $\epsilon =$  {v}")
    print("    [4] Ввести своё значение")

    while True:
        try:
            choice = int(input("\n Ваш выбор (1-4): ").strip())
            if choice in presets:
                return presets[choice]
            if choice == 4:
                val = float(input("    Введите  $\epsilon$  (например, 0.005): ").strip())
                if val > 0:
                    return val
                print("     $\epsilon$  должна быть положительной.")
            else:
                print("    Введите число от 1 до 4.")

```

```

        except ValueError:
            print(" Неверный ввод, попробуйте снова.")

def ask_show_iterations() -> bool:
    ans = input("\n Показать таблицу итераций удвоения n? (д/н): ").strip().lower()
    return ans in ("д", "y", "да", "yes")

def run_all(a: float, b: float):
    """Прогон по всем трём ε из задания."""
    for eps in [0.01, 0.001, 0.0001]:
        sep("=")
        print(f" ВЫЧИСЛЕНИЯ ДЛЯ ε = {eps}")
        results = compute(a, b, eps)
        print_report(results, eps, a, b)

def main():
    a = 0.0
    b = math.pi

    sep("=")
    print(" ЛАБОРАТОРНАЯ РАБОТА №6")
    print(" Численное интегрирование – прямоугольники, трапеции, Симпсон")
    sep("=")
    print(f"\n f(x) = x2 · exp(-x2), ∫0π f(x) dx\n")

    print(" Режим работы:")
    print(" [1] Выбрать одно значение ε (интерактивный режим)")
    print(" [2] Прогон по всем ε из задания: 0.01 / 0.001 / 0.0001")

    while True:
        try:
            mode = int(input("\n Ваш выбор (1/2): ").strip())
            if mode in (1, 2):
                break
            print(" Введите 1 или 2.")
        except ValueError:
            print(" Неверный ввод.")

    if mode == 2:
        run_all(a, b)
        return

    # ——— Интерактивный режим ———
    eps = choose_epsilon()
    # ← сначала выбираем

```

ε

```

sep("—")
print(f"  Запуск вычислений при  $\epsilon = \{eps\}$  ...")
sep("—")

results = compute(a, b, eps)  # ← пункт 3: удвоение n
по Рунге

if ask_show_iterations():
    print_iterations(results)  # ← промежуточные таб
лицы

print()
print_report(results, eps, a, b)  # ← пункты 4 и 5

print()
again = input("  Повторить с другим  $\epsilon$ ? (д/н):
").strip().lower()
if again in ("д", "y", "да", "yes"):
    main()

if __name__ == "__main__":
    main()

```